A smaller clausal presentation of iPRA

Root-free evaluation, a uniform companion, and checked proof translations

Prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

We combine the repository's clause presentation with its elimination of the direct equality-composition case. The resulting raw calculus has terms $x, 0, S(t), I(n, a; u.h)$, formulas $s = t$ and $\mathsf{D}_{x,y}B$, and five instruction patterns: reflexivity, two-case evaluation, clause use, paired elimination, and paired introduction. It has ordinary capture-avoiding substitution. The complete standalone checker has 2,956 normalized bytes, 837 significant tokens, and 977 AST nodes. Its matched control has 3,003 bytes, 842 tokens, and 984 AST nodes; the exact incoming raw-clausal checker has 3,046 bytes. The arithmetic deletion alone accounts for 47 bytes, separately from an ordinary helper cleanup. Every open sequent is preserved literally. A proof with N nodes and k direct composition nodes translates to a proof with exactly $N + 2k$ nodes, without duplicating its premise proofs.

A zero-normal companion uses one successor-evaluation schema and the same five instruction patterns. A fixed-zero-base introduction guard removes the literal-witness test. Its checker has 2,948 bytes, 852 tokens, and 987 AST nodes; it uses more tokens and AST nodes than the raw proposal. Both ordinary realizations are bi-interpretable and equiconsistent with full first-order intuitionistic primitive recursive arithmetic. Both numerical domains are whole, equality is actual equality, and the numerical composites have graph $x = y$. The component mathematical reductions are inherited; the contribution is their explicit integration, comparison, and executable proof translations. No global simplicity minimum, new compact-codec record, or new Lean/Rocq verification is claimed.

Contents

1 Result and comparison target **3**

2 The complete raw axiomatization **3**
2.1 Numerical terms, formulas, and clauses 3
2.2 All primitive rules 4

3 Deleting the direct equality case **5**

4 Recovering arithmetic and logic **6**
4.1 Equality and the iterator equations 6
4.2 All numerical cases and equation induction 6
4.3 An explicit ordinary logical vocabulary 7

5 A companion with one arithmetic evaluation schema 8
5.1 Canonical syntax and substitution . . . 8
5.2 A fixed zero-base test still recovers all induction . . . 9
5.3 Why normalization preserves the whole theory . . . 9

6 Full iPRA and both interpretation composites 10
6.1 The ordinary theory presented . . . 10
6.2 The inherited arithmetic bootstrap . . . 11
6.3 The composites are internally the identity . . . 11

7 Complete checkers and measured costs 12
7.1 Concrete representation and scope invariants . . . 12
7.2 Matching measurements . . . 13

8 Verification, reproducibility, and limits 13

9 Further research questions 14

Attribution and references 15

Inherited bridge proofs 16

A Complementary introductions with one local assumption 16
A.1 Complete native syntax and rule inventory . . . 16
A.2 Exact equivalence and the unused-witness construction . . . 17
A.3 Induction and numerical cases separate cleanly . . . 18
A.4 The entire ordinary logical interface . . . 19
A.5 Whole-domain realizations and both formula composites . . . 20
A.6 What the theorem does and does not claim . . . 21

B Arithmetic strength and both ordinary interpretation composites 21
B.1 The arithmetic dependency order . . . 22

C Internal scalar predecessor bootstrap 23
C.1 Elementary terms and two finite cycles . . . 23
C.2 A ramp with a global scaling identity . . . 24
C.3 A comparator from scalar iteration . . . 25
C.4 Saturated iteration gives predecessor . . . 25
C.5 Dependency boundary . . . 27

D From literal-equation induction to full intuitionistic PRA 27
D.1 The exact hypotheses . . . 27
D.2 Constructor separation and two elementary tests . . . 28
D.3 Four elementary uses of equation induction . . . 29
D.4 An equation invariant that reflects actual equality . . . 30
D.5 Compilation of all quantifier-free formulas . . . 31
D.6 Recovery of induction and the full theory . . . 32

E Pairing, primitive recursion, and internal numerical composites **32**
E.1 Pairing from a bounded square-root search 32
E.2 Compiling each primitive recursion 33
E.3 Why both numerical round trips are internal 34

F Complete raw root-free checker **35**

G Complete zero-normal companion checker **37**

1 Result and comparison target

The raw calculus $\mathcal{R}$ in Section 2 is the recommended profile when ordinary substitution and the token/AST coordinates matter most. It gives a strict improvement in all three reported source-size coordinates over a matching implementation of the current raw clausal calculus. The zero-normal calculus $\mathcal{Z}$ in Section 5 has the smallest byte count among the selected profiles, one arithmetic evaluation schema, and a fixed-zero-base introduction test. Its normalizing substitution is explicitly counted. These are two complete programs, not three coordinatewise minima assembled from unrelated programs.

The comparison is pinned to ProveIt commit

`831081033253cca2115196cf86bb99c77ed2aeae`.

The reviewed subtree is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. Its README describes fifty filed research packages and a Lean formalization. Five additional ZIP archives were present in its `docs/incoming` directory. In particular, the following results already existed:

- the Eq/D clause interface, removing the separate Q formula constructor;
- three-case computational equality, which combines computation and transport;
- removal of the direct equality-composition branch by a zero-iterator frame;
- zero-normal terms with reflexivity and one successor-evaluation schema;
- a paired-introduction guard based on the reflexivity of its zero instance.

Thus the present package does not claim priority for any of these component ideas. Its raw calculus combines the first and third results; its canonical companion combines the first and fourth. The source ledger in `evidence/provenance.json` identifies the actual comparison material, including the incoming archives. The main arithmetic bridge is included in the appendices, with its upstream attribution.

The word “simpler” here has a precise, limited meaning: a smaller complete standalone checker in the stated matching family, with a strictly reduced set of arithmetic cases. There is no proved lower bound over all possible presentations, languages, encodings, or host languages. The much smaller compressed checker profiles already listed in the repository remain smaller.

2 The complete raw axiomatization

2.1 Numerical terms, formulas, and clauses

There is one numerical sort. Terms are finite syntax trees

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u in h is bound in an iterator. The counter n and initializer a are outside that binder. Bodies may contain fixed numerical parameters; there are no variables over functions and no universal evaluator. Substitution is ordinary capture-avoiding substitution, including under iterator binders. Put $1 = S0$.

Formulas and retained context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Both x and y are bound in $\mathsf{D}_{x,y}A$, and are distinct. A context is a finite list of clause occurrences. The clauses are proof-context data, not additional objects in the numerical domain. Their ordinary readings are

$$(\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y\, B^\circ(x,y)\big), \qquad \langle A; y.B\rangle^\circ = A^\circ \to \forall y\, B^\circ.$$

An ordinary formula assumption F abbreviates the constant clause $\langle H; v.F\rangle$, where H is the formula $0 = 0$ and v is unused. The designated contradiction is $0 = S0$.

2.2 All primitive rules

The following display is the entire primitive inventory.

Reflexivity.

$$\frac{}{\Gamma \vdash t = t} \quad (\mathsf{R}).$$

Evaluation. There are two arithmetic cases of one evaluator instruction:

$$\frac{\Gamma \vdash a = t \qquad \Gamma \vdash I(0,a;u.h) = r}{\Gamma \vdash r = t} \quad (\mathsf{V}_0), \tag{V0}$$

$$\frac{\Gamma \vdash I(n,a;u.h) = t \qquad \Gamma \vdash I(Sn,a;u.h) = r}{\Gamma \vdash r = h[t/u]} \quad (\mathsf{V}_S). \tag{VS}$$

The repeated terms in these patterns must match syntactically, up to a fixed implementation of bound-variable renaming. In particular no test of semantic equality is hidden in the side conditions. The body h is read from the second equality premise, not supplied as a separate transport template.

Clause use.

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad (\mathsf{U}), \qquad \langle A; y.B(y)\rangle \in \Gamma. \tag{U}$$

Paired elimination.

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(b,0); y.B(b,y)\rangle \vdash F}{\Gamma \vdash F} \quad (\mathsf{E}). \tag{E}$$

The eigenparameter b is fresh for Γ, F, and all passive matrix parameters.

Paired introduction.

$$\frac{\Gamma, B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}\ (\mathsf{I}), \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases} \tag{I}$$

The eigenparameter z is fresh for Γ, t, and all passive matrix parameters. The displayed local formula assumption is its constant clause, as defined above, and is discharged. An equation under a D binder is not a literal equation.

There are five instruction tags: $\mathsf{R}, \mathsf{V}, \mathsf{U}, \mathsf{E}, \mathsf{I}$. If each arithmetic row of V is counted separately, the display has six schemas. The introduction also has an explicit mode test. These conventions are stated to avoid obtaining an apparent reduction merely by renaming several rules as one. The real deletion is the third arithmetic case described next. No additional computation, equality, logical, successor, induction, or structural axiom is primitive.

3 Deleting the direct equality case

Let $\mathcal{E}$ be the incoming raw clausal calculus. It has exactly the same syntax and rules as $\mathcal{R}$, with one extra evaluator case

$$\frac{s = t \qquad s = r}{r = t}. \tag{VId}$$

All contexts in this section may contain arbitrary retained clauses.

Lemma 3.1 (An iterator frame recovers direct composition)**.** *Rule* (VId) *is derivable using only reflexivity and* (V0)*. Its two input proofs each occur once.*

Proof. Given $p : s = t$ and $q : s = r$, put $W(s) = I(0, s; u.0)$. Reflexivity gives $W(s) = W(s)$. Apply (V0) first to q and that reflexivity:

$$W(s) = r.$$

Apply it again to p and this equation, giving $r = t$. In proof notation the construction is

$$\mathsf{V}_0(p, \mathsf{V}_0(q, \mathsf{R}(W(s)))).$$

No induction, clause manipulation, successor computation, numerical cases, or prior symmetry principle is used. □

Theorem 3.2 (Exact preservation of every open sequent)**.** *For every raw clause context* Γ *and raw formula* A*,*

$$\Gamma \vdash_{\mathcal{E}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{R}} A.$$

Both transformations are effective on finite proof trees. Numerical terms, formulas, contexts, and free numerical parameters are unchanged.

Proof. Every $\mathcal{R}$ rule is an $\mathcal{E}$ rule. In the other direction, induct on the $\mathcal{E}$ derivation and replace each use of (VId) by Lemma 3.1. All other nodes are copied with their already translated premise proofs. No variable is bound or freed by the replacement, so all introduction and elimination freshness conditions remain valid. □

Corollary 3.3 (Exact proof-node overhead)**.** *If the input tree has* N *inference nodes and* k *uses of* (VId)*, the displayed translation has exactly* $N + 2k$ *nodes. In particular its size in inference nodes is at most* $3N$*.*

Proof. Each replaced node becomes two evaluation nodes and one new reflexivity leaf. Both original premise subtrees are inserted once. All other nodes are unchanged. □

This is not a bound on JSON byte length: the new frame contains a copy of the source term. The shipped compiler preserves the end formula literally and checks its result using the complete target checker. Its reverse inclusion is the identity on proof syntax.

4 Recovering arithmetic and logic

Theorem 3.2 already transfers the earlier derivability and interpretation results. We give the principal derivations explicitly so the new inventory is independently understandable.

4.1 Equality and the iterator equations

Write $\mathrm{Root}(p, q)$ for Lemma 3.1. Given $p : s = t$, its symmetry is $\mathrm{Root}(\mathsf{R}(s), p) : t = s$. Given additionally $q : t = v$, transitivity is $\mathrm{Root}(q, \mathrm{Sym}(p)) : s = v$. The two computation equations are evaluations of reflexivities:

$$\mathsf{V}_0(\mathsf{R}(a), \mathsf{R}(I(0, a; h))) : I(0, a; h) = a,$$

$$\mathsf{V}_S(\mathsf{R}(I(n, a; h)), \mathsf{R}(I(Sn, a; h))) : I(Sn, a; h) = h[I(n, a; h)/u].$$

Lemma 4.1 (Congruence for any fixed term template)**.** *From $s = t$, derive $h(s) = h(t)$ for every finite template $h(u)$, including multiple occurrences and occurrences under nested iterator binders.*

Proof. Let $A_s = I(0, s; u.h)$ and $B_s = I(1, s; u.h)$. The following four evaluations are all primitive cases, with no appeal to a derived transport rule:

$$\begin{aligned} z_s &= \mathsf{V}_0(\mathsf{R}(s), \mathsf{R}(A_s)) && : A_s = s, \\ c_s &= \mathsf{V}_S(z_s, \mathsf{R}(B_s)) && : B_s = h(s), \\ z_{s,t} &= \mathsf{V}_0(p, \mathsf{R}(A_s)) && : A_s = t, \\ \mathsf{V}_S(z_{s,t}, c_s) & && : h(s) = h(t). \end{aligned}$$

Substitution into the iterator body is precisely the primitive substitution in (VS), so the argument applies to the full term grammar. □

Congruence followed by Root gives one-term crossed transport, $s = t, h(s) = r \Rightarrow r = h(t)$. Two such transports give transport in an arbitrary equation template.

4.2 All numerical cases and equation induction

Ordinary assumption use is derivable: prove H by reflexivity and apply U to its constant clause. Weakening, fresh renaming, and ordinary-assumption cut are admissible by induction on derivations. For cut, each U that selects the distinguished constant-clause occurrence is replaced by the supplied proof; all other nodes are translated recursively after freshening eigenparameters. These arguments do not assume that arbitrary substitution preserves the introduction mode.

All case introductions are derivable:

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}. \qquad \text{(Case)}$$

Only an equation at the literal witness 0 is missing from the primitive inventory. For that instance use $c = I(0, 0; u.0)$ as the witness. It is syntactically different from 0 and is provably equal to 0. Equation transport and ordinary cut move the assumed and resulting instances between c and 0. Primitive introduction at c is in case mode and gives the required conclusion.

Arbitrary capture-avoiding raw substitution is now admissible. Induct on the derivation after freshening eigenparameters. A progressive instance keeps its equation root and literal-zero witness. A Case instance can change mode only when its substituted witness becomes literal zero; use the unrestricted derived Case rule in that event. All other rules commute with substitution. Thus the ordinary logical macro proofs below may substitute parameters without an unproved mode-stability assumption.

Transport through a compound formula follows by external induction on its syntax. For $\mathsf{D}_{x,y}B(v, x, y)$, eliminate the old paired formula at a fresh b, introduce the new one by Case at b, transport its assumed zero instance backwards, use the opened old clause at Sz, and transport the successor instance forwards. The recursive transports concern the strictly smaller matrix. This establishes full formula transport without arithmetic induction.

For an unused w, write $q_yB(y) = \mathsf{D}_{w,y}B(y)$. Case gives its introduction from $B(0) \vdash B(Sz)$; E and U give its specialization from q_yB and $B(0)$ to $B(t)$. A retained clause $\langle B(0); y.B(y)\rangle$ itself proves q_yB by Case. These are the same three interfaces as the old native Q.

For a literal equation $E(y)$, primitive I with unused witness 0 gives q_yE from $E(z) \vdash E(Sz)$. Therefore

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

is derived. Likewise Case, E, and U give numerical cases for arbitrary formulas from their zero and successor branches. Progress has not been granted to arbitrary quantified formulas.

4.3 An explicit ordinary logical vocabulary

The entire logical interface is recalled here; the detailed inherited proofs are in Appendix A. Define $C(v, a, b) = I(v, a; u.b)$ with the bound u unused. Then $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define a gate on formulas by

$$G_v(s = t) = (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) = \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. The formulas G_0A are theorems, and $G_{Sv}A$ is interderivable with A. Padding is $A \dashv\vdash \mathsf{D}_{u,y}G_yA$, with u, y fresh.

Select between equations by

$$M_v(s = t, r = w) = (C(v, s, r) = C(v, t, w)).$$

Align paired binders recursively; if one argument is an equation and the other is D-headed, pad the equation using G_y under that binder. Explicitly, with freshly aligned x, y,

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &= \mathsf{D}_{x,y}M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y}B) &= \mathsf{D}_{x,y}M_v(G_yE, B), \\ M_v(\mathsf{D}_{x,y}A, E) &= \mathsf{D}_{x,y}M_v(A, G_yE). \end{aligned}$$

This recursion decreases the maximum binder depth. It gives $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$. Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. The following are finite macros, not extra primitive

symbols:

$$\begin{aligned} A \Rightarrow_* B &= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &= (0 = S0). \end{aligned} \tag{Logic}$$

Unused coordinates and all fresh binders are chosen capture-free. Implication follows from the selector endpoints; the existential from the gate; conjunction uses $p(1) = 1$ and $p(2) = 0$; disjunction uses controls 0 and 1 and numerical cases. Universal elimination combines the zero conjunct and the successor instances. Under $0 = S0$, selector transport gives any equation, and induction on formula syntax gives explosion for every formula. These constructions establish intuitionistic logic, without excluded middle for arbitrary formulas.

5 A companion with one arithmetic evaluation schema

The raw system gives the cleanest exact comparison. The companion $\mathcal{Z}$ trades a normalizing substitution operation for one arithmetic schema and a fixed-zero-base introduction test.

5.1 Canonical syntax and substitution

Let nf recursively contract every iterator whose counter is the literal 0:

$$\mathrm{nf}(I(n, a; u.h)) = \begin{cases} \mathrm{nf}(a), & \mathrm{nf}(n) = 0, \\ I(\mathrm{nf}(n), \mathrm{nf}(a); u.\, \mathrm{nf}(h)), & \text{otherwise.} \end{cases}$$

It acts structurally on variables, 0, successor, formulas, and clauses. Numerical terms in $\mathcal{Z}$ are exactly the fixed points of nf. Every substitution is followed by nf; this includes contractions first exposed by opening a binder. Successor-counter iterators are not reduced by this normalizer.

Use the same clause rules and reflexivity as before, all with this canonical substitution. Replace both raw evaluation rows by the single schema

$$\frac{\Gamma \vdash \mathrm{nf}(I(n, a; u.h)) = t \qquad \Gamma \vdash I(Sn, a; u.h) = r}{\Gamma \vdash r = \mathrm{nf}(h[t/u])}. \tag{UE}$$

There is no equality-composition case and no zero-evaluation case. Replace the earlier introduction by

$$\frac{\Gamma, B(t, \eta) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}, \qquad \eta = \begin{cases} z, & \mathrm{nf}(B(t, 0)) \text{ is literally } (0 = 0), \\ 0, & \text{otherwise.} \end{cases} \tag{I0}$$

Freshness is unchanged. The test is one exact comparison of the computed base with a fixed formula. It automatically excludes compound matrices, because numerical substitution and zero normalization preserve formula roots. There is no separate equation-root or witness test. The total inventory is five instruction tags: Refl, UE, U, E, and I0. Zero computation has moved into syntax; its implementation is counted in the checker.

Lemma 5.1 (Canonical equality bootstrap)**.** *Reflexivity and UE derive direct composition, every numerical congruence, and successor computation in $\mathcal{Z}$.*

Proof. For direct composition from $p : s = t$ and $q : s = r$, put $K(s) = I(1, s; u.u)$. Since $\mathrm{nf}(I(0, s; u.u)) = s$,

$$\mathrm{UE}(p, \mathrm{UE}(q, \mathsf{R}(K(s))))$$

proves $r = t$. The two UE instances have the same identity body. Symmetry and transitivity follow as in the raw calculus.

For congruence put $J = I(1, s; u.h)$. UE on reflexivities gives $J = \mathrm{nf}(h[s/u])$; UE on p and that equation gives $\mathrm{nf}(h[s/u]) = \mathrm{nf}(h[t/u])$. Successor computation is UE on the two appropriate reflexivities. Thus no transport rule is presupposed by the evaluator's equality bootstrap. □

5.2 A fixed zero-base test still recovers all induction

Ordinary assumption use, weakening, and ordinary-assumption cut are proved as before. Every Case instance is especially simple to recover. If its base is not literally $0 = 0$, I0 is already Case. If the base is $0 = 0$, cut its reflexivity proof into the supplied Case branch, then weaken that branch by the progressive assumption and apply I0. No artificial witness equal to zero is needed. Consequently all three q interfaces and numerical case reasoning are available before any equation induction is recovered.

Here is the full induction recovery, including the point where an arbitrary equation is changed into one with the required base. Suppose $E(y)$ is $s(y) = t(y)$ and a step proof $\Gamma, E(z) \vdash E(Sz)$ is supplied. To introduce $q_y E(y)$ by Case, retain $E(0)$ and aim to prove $E(Sw)$ for fresh w. Put $s_0 = s(0)$, $t_0 = t(0)$ and

$$H(y) : \quad C(y, 0, s(y)) = C(y, 0, t(y)).$$

In canonical syntax $H(0)$ is literally $0 = 0$. Consider the two decoders

$$\begin{aligned} F_s(y) : &\quad C(y, s_0, C(y, 0, s(y))) = s(y), \\ F_t(y) : &\quad C(y, s_0, C(y, 0, t(y))) = t(y). \end{aligned}$$

Their zero instances are respectively reflexivity and the retained $E(0)$. Every successor instance is an equality obtained by two selector computations. Thus Case gives each conditional closure, and its supplied base lets the derived q-specialization (E followed by U) give $F_s(z)$ and $F_t(z)$.

Under $H(z)$, congruence for $v \mapsto C(z, s_0, v)$, together with the two decoder equations, gives $E(z)$. Cut this into the original step proof to obtain $E(Sz)$, then use selector computation to infer $H(Sz)$. I0 is therefore applicable to the H matrix with an unused existential coordinate: its zero instance is exactly $0 = 0$. It gives $q_y H(y)$. Reflexivity supplies its base, and derived q-specialization at Sw gives $H(Sw)$, hence $E(Sw)$ by computation. The outer Case now proves $q_y E(y)$.

This derives unrestricted equation progress. With a supplied base it derives EI. The proof uses no equation induction before I0 is applied: the decoders use Case, and their equality reasoning comes from Lemma 5.1. Conversely, when the guard is true, equation progress first gives $q_y B(t, y)$. Its reflexive zero base and derived specialization give each successor instance; Case at the original witness t then gives the I0 conclusion. When the guard is false, I0 already is Case. Hence the fixed-base and separate Case/progress presentations have exactly the same consequences. This guard and its symmetric lift are inherited from the incoming reflexive-base report; the integration here uses them with clausal single evaluation.

5.3 Why normalization preserves the whole theory

The necessary inherited normalization theorem says that the full native calculus proves a raw sequent exactly when its zero-normal counterpart proves the normalized sequent. The Lean source

contains the corresponding theorem `nzprf_iff_nprf`; the incoming single-evaluator report gives the Refl/UE equality presentation used here.

There are two substantive points behind that theorem. First, $\mathrm{nf}(\mathrm{nf}(t)\sigma) = \mathrm{nf}(t\sigma)$, and canonical substitution is compatible with composition. Second, the ordinary equality $t = \mathrm{nf}(t)$ must also hold when normalization changes an iterator's body. Given pointwise equal bodies and equal seeds, equation induction on the iterator counter proves equality of the two iterators: the base is seed equality, and the step uses the supplied body equality and the induction hypothesis. External induction on term syntax now proves $t = \mathrm{nf}(t)$, using ordinary term congruence also to change the counter from n to $\mathrm{nf}(n)$.

The exact-zero guard behaves well under substitution in one direction: if the base is literally $0 = 0$, it stays so. A formerly nonzero base can become $0 = 0$, changing a primitive Case instance into a progressive one. That instance is repaired by the derived unrestricted Case rule above. Thus arbitrary substitution is admissible, although keeping every original rule tag is not a correct proof transformation. Equivalently, pass through the calculus with separate Case and equation-progress rules and recover I0 afterwards. This is part of the mathematical specification, not a missing checker side condition.

Finally replace native $Q_y B$ by $\mathsf{D}_{w,y} B$ with w unused. In the other direction, a clause introduced by E is represented by its Q assumption, whereas an ordinary constant clause is represented by its formula assumption. U translates respectively to QE or Hyp. The three closure interfaces from Section 4 prove the forward direction. Occurrence annotations distinguish clauses of different origins even if their printed values coincide. These translations preserve normal terms and the exact-zero guard: they leave equations unchanged, and a compound formula stays compound.

For an arbitrary initial retained clause $\langle A; y.B\rangle$, rather than one generated by a proof rule, use its native representation $A \Rightarrow_* \forall_* y\, B$. The derived implication and universal eliminations simulate U. This extends the ordinary reading beyond the origin-annotated generated contexts used in the executable translators.

Theorem 5.2 (Canonical companion)**.** *The ordinary realization of $\mathcal{Z}$ is bi-interpretable and equiconsistent with that of $\mathcal{R}$. Its numerical interpretations use the whole domain and actual equality. The raw-to-canonical numerical term composite is provably equal to the identity; on canonical syntax the other composite is the identity.*

Proof. Compose the inherited zero-normalization equivalence, the canonical equality bootstrap, and the clause/Q equivalence just described. The integration replaces the formula-context interface by clauses, retains UE, and uses the inherited fixed-zero-base introduction with the complete progress recovery just proved. All equality bootstrap proofs work under arbitrary retained clauses. Formula transport and binder congruence extend the numerical normalization identity through formulas. □

6 Full iPRA and both interpretation composites

6.1 The ordinary theory presented

Let T_I be intuitionistic first-order logic with actual equality and genuine falsity. Its function symbols are 0, S, and the staged scalar iterators: for each fixed earlier finite term $h(u, \vec{p})$, add $i_h(n, a, \vec{p})$ with

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Its remaining nonlogical principles are numerical coverage, separation, and contextual induction on literal equations:

$$\forall x\, (x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot, \qquad \text{EI}.$$

The base logic can be the usual intuitionistic natural deduction rules for $\bot, \wedge, \vee, \to, \forall, \exists$ with reflexivity and equality substitution. Its native realization is completely specified by Section 2; no unspecified ordinary logic is required by the standalone checker.

The ordinary readings of Eq, D, and clauses were given above. Under them every rule is sound in T_I: Case uses coverage; the progressive branch uses equation induction under the conditional base. Conversely, the explicit logical macros give all intuitionistic rules, EI gives equation induction, and numerical cases give coverage. Separation translates to the identity implication $(0 = S0) \Rightarrow_* (0 = S0)$.

The falsity bridge must be explicit. Computation and positive equality alone admit a singleton model; they cannot silently be identified with a theory whose logical falsity has its usual interpretation.

6.2 The inherited arithmetic bootstrap

The included arithmetic proof proceeds in the following order:

1. Construct a scalar term P with $P0 = 0$ and $P(Sn) = n$. This also proves successor injectivity.
2. Construct truncated subtraction and a faithful term test for actual numerical equality, valid at arbitrary model elements.
3. Turn each fixed quantifier-free formula into a characteristic equation. Its equation induction gives quantifier-free induction.
4. Construct pairing and projections by bounded search, and compile every fixed general primitive recursion into scalar iteration.

The order matters: pairing and equality reflection are not assumed when predecessor is constructed. The full inherited arguments appear in Appendices C–E. They are not claimed as new here.

For clarity, let π, L, H be the constructed pairing and projections. A target primitive recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

is compiled by

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big),$$
$$f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$, and the projection equations prove the desired defining equations. This is an external effective operation on fixed finite descriptions, not an internal universal primitive-recursive evaluator.

6.3 The composites are internally the identity

Interpret an iterator in ordinary iPRA by primitive recursion whose update ignores its counter. Interpret each fixed iPRA function description in T_I by the preceding compiler. In both directions the domain formula is $x = x$, and equality remains $x = y$.

Equation induction proves uniqueness for each fixed scalar iteration and each fixed ordinary primitive recursion. External induction on a finite description shows that its double translation satisfies its own original defining equations, hence is internally equal to the original function. The graph $x = y$ is therefore an internally provable total bijection preserving every operation in either numerical composite. This is a statement about all numerical elements, not just standard numerals.

For formulas, the crucial composite is

$$\mathsf{D}_{x,y}B \quad \dashv\vdash \quad \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)\big). \tag{DC}$$

Forward, eliminate the paired formula at a fresh b, assume its base, and use the resulting clause at an arbitrary numerical parameter; then introduce the universal, implication, and existential macros. Backward, eliminate the existential and introduce D by Case, using the supplied conditional universal at the successor. The ordinary interpretations of the macros in (Logic) have their usual round trips by the selector and gate endpoint proofs. Structural induction on formulas completes both formula composites.

Theorem 6.1 (Required equivalence)**.** *The ordinary realizations of $\mathcal{R}$ and $\mathcal{Z}$ are each bi-interpretable and equiconsistent with full first-order* iPRA*. Both numerical domains are whole; interpreted equality is actual equality; both numerical composite isomorphisms have graph $x = y$.*

Proof. The preceding internal bridge gives $T_I \simeq$ iPRA. The inherited native logical bridge gives $\mathcal{E} \simeq T_I$. Theorem 3.2 identifies every $\mathcal{R}$ sequent with the corresponding $\mathcal{E}$ sequent on unchanged syntax. The canonical companion theorem provides the remaining equivalence. The explicit falsity correspondence yields equiconsistency in both directions. □

7 Complete checkers and measured costs

Each selected checker is one standalone Python module using only `json` and `sys`. Its public input is exactly

```
{"goal": <closed Eq/D formula>, "proof": <finite proof tree>}
```

Acceptance prints `ACCEPT` and exits with status zero. Rejection prints `REJECT` and exits nonzero. Checks do not rely on Python `assert`; optimized execution has the same intended acceptance relation. The abstract calculus concerns arbitrary finite derivations; ordinary finite-machine memory and recursion limits apply to the programs.

7.1 Concrete representation and scope invariants

Zero is integer 0. Free eigenparameters are negative levels, and bound variables use positive de Bruijn indices. Successor is `[0,t]`, iteration is `[n,a,h]`, equality is `[s,t]`, and a paired formula is `[B]`. The iterator body binds one index; a paired formula binds two. An opening replaces index $d+1$ at binder depth d and decreases larger indices by one. Its replacement is a well-scoped term with no unbound positive indices, so negative free levels avoid capture without shifting external parameters.

Terms, formulas, rule tags, rule arities, context indices, witnesses, conclusions, and eigenparameter scopes are checked. Boolean JSON values are not accepted as integers. Validation concerns the decoded JSON object; the standard parser uses last-key semantics for duplicate textual keys. Each derivation rule preserves the typing invariant of its inferred formula. The evaluator can therefore inspect the shape of an already inferred equality without repeating a full validator. A purported successor counter of any other shape is rejected. The normal companion additionally rejects zero-counter iterators anywhere in supplied term syntax and contracts them whenever opening creates one.

The finite CLI takes closed theorems from the empty context. The recursive kernel implements local open sequents and clauses, which are used in every introduction and elimination. The mathematical equivalence theorem is for all open sequents, not merely the finite test inputs.

7.2 Matching measurements

Measurements use CPython 3.12.14. Remove the module docstring, parse the entire source, print it with `ast.unparse`, and append one line feed. Significant tokens exclude indentation, line-ending, end-marker, encoding, and comment tokens. AST nodes count the entire parsed module. No imported project helper or validator is excluded. Gzip uses level 9 and timestamp zero, and is informational only.

Complete checker	Bytes	Tokens	AST	Gzip
Exact incoming raw clausal	3046	853	993	990
Matched raw control	3003	842	984	978
Raw root-free proposal	**2956**	**837**	**977**	**962**
Zero-normal control with root case	2993	864	1006	985
Zero-normal UE, witness guard	2948	853	992	971
Zero-normal UE, fixed-zero guard	**2948**	**852**	**987**	**967**
Tagged native normal comparator	3011	979	1047	965

The matched raw control merely inlines a successor-constructor helper that was called once. That cleanup is shared with the proposal. Deleting the direct arithmetic branch then saves exactly 47 normalized bytes (1.565%), 5 significant tokens, and 7 AST nodes. Relative to the unchanged incoming file the total savings are 90 bytes (2.955%), 16 tokens, and 16 AST nodes; 43 of those bytes are the helper cleanup.

The witness-guard canonical intermediate saves 45 bytes, 11 tokens, and 14 AST nodes against its matching root-retaining control. Replacing its witness guard by the fixed-zero-base guard then saves one token and five AST nodes at unchanged byte count. The selected fixed-zero companion is 8 bytes smaller than the raw proposal but has 15 more tokens and 10 more AST nodes. Consequently neither selected checker dominates the other on all three coordinates.

The earlier tagged raw two-case checker has 2,996 bytes, 954 tokens, and 1,019 AST nodes. The present raw clausal checker is smaller in all three, but this comparison changes the logical interface and variable representation; it is not the isolated 47-byte arithmetic ablation. The repository's historical 516-byte compact profile and smaller validated compressed encodings are not beaten. We claim a transparent complete checker improvement, not a global source-size frontier.

8 Verification, reproducibility, and limits

The raw proposal was checked on 33 positive and 27 negative certificates, including 120 ordinary and optimized command-line runs. All 33 positive proofs passed the exact-sequent compiler, the reverse inclusion, and the $N+2k$ node-count check. The examples include used induction hypotheses, closure specialization, repeated-variable congruence, zero-source transport, capture-avoiding witnesses, and nested compound binders.

The witness-guard canonical intermediate was checked on 15 positive and 27 negative certificates, with 84 ordinary and optimized command-line runs. Each positive proof was translated into the independently implemented inherited tagged six-rule canonical checker and accepted. An additional 6,000 opening comparisons checked the negative-level representation against that independent de Bruijn implementation. They include contractions created by opening as well as binder-capture cases.

The selected fixed-zero companion was separately checked on 17 positive and 28 negative certificates, with 90 ordinary and optimized command-line runs. All 17 positive proofs were

translated into the inherited seven-rule zero-normal checker, which uses separate computation and transport rules, and accepted. Additional cases use progress at a nonzero witness, cases at witness zero with base $1 = 1$, and rejection of progress when the base is not $0 = 0$. Thus the changed guard is tested directly.

Invalid cases include escaping eigenparameters, an unshifted or unbound index, invalid iterator counters, forged evaluation sources, a wrong clause antecedent, Boolean indices, compound-progress attempts, malformed proof arities, and extra top-level fields. There were zero unexpected outcomes in the recorded runs.

These are implementation checks. The universal correctness statements are the written derivation transformations and inherited internal arithmetic proofs. This package does not claim a new Lean or Rocq proof. The upstream README reports a compiled Lean development, but it was not rebuilt during this work. The selected raw and canonical Python profiles have different syntactic contracts; no general raw-to-canonical proof compiler is advertised. The executable raw translation stays within raw syntax, and the canonical translation targets the separate inherited canonical checker.

From the extracted package root, run

```
python3 reproduce.py
python3 code/raw_two_case.py < code/raw_certificates/used_induction_hypothesis.json
python3 code/zero_base.py < code/zero_base_certificates/used_equation_induction.json
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The report's distributed `report.tex` is standalone: its inherited appendices and code listings are embedded. The editable assembly source and source fragments are also provided. The checks require only Python's standard library; the PDF rebuild requires a normal TeX Live installation.

9 Further research questions

1. **Formalize the exact integration.** Add a clausal root-free calculus to the existing Lean development and prove the sequent equivalence by the three-node frame replacement. This is a small extension relative to the arithmetic bridge, but the context clauses and their occurrence annotations should be represented explicitly.

2. **Can the clean rule basis improve the compact frontier?** The present matched strict improvement does not beat the old compressed codecs. A meaningful further reduction must count grammar validation, normalization if used, and the complete public interface.

3. **Can introduction be made both simpler and substitution-stable?** The raw profile retains an explicit literal-witness test; the canonical profile selects progress by the exact zero base. The existing diagonal guard is stable in both directions under substitution, while the fixed-zero guard may change from false to true. Can this remaining proof-translation complication be removed at equal or smaller cost?

4. **Can proof certificates share the iterator frames?** The $N + 2k$ theorem controls tree nodes. DAG certificates could share repeated frames and reflexivities, but their reference validation and acyclicity checks could erase the verifier saving.

5. **Can a narrower uniqueness rule replace equation induction?** The incoming scalar-uniqueness package establishes a full scalar uniqueness bridge. The separate successor-rigidity report reaches zero-equation induction and leaves full equality reflection open. Neither fact licenses removing the equation-progress restriction here.

6. **What cost model should define a genuine minimum?** Rule rows, trusted normalization, parser scope, certificate format, source bytes, AST nodes, and running time measure different things. A formal optimization problem needs to fix these choices before an absolute simplest-presentation claim can have a determinate meaning.

Attribution and references

The clause interface, scalar bridge, zero normalization, witness-directed introduction, and iterator-frame equality reductions are inherited from ProveIt contributors. Code and reproduced mathematical source fragments retain the accompanying MIT No Attribution license. The new work is the combined specification, matching implementation, proof compilers, comparison, and validation described above.

References

[1] ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic.* Pinned snapshot `831081033253cca2115196cf86bb99c77ed2aeae`. https://github.com/VladimirReshetnikov/ProveIt/tree/831081033253cca2115196cf86bb99c77ed2aeae/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt contributors. *Five rules for iPRA with ordinary substitution.* `docs/incoming/ipra_raw_clausal_2026-11-11.zip`, same snapshot.

[3] ProveIt contributors. *Equality from iteration.* `docs/incoming/ipra_equality_from_iteration_2026-11-11.zip`, same snapshot.

[4] ProveIt contributors. *Five rules for intuitionistic primitive recursive arithmetic.* `docs/incoming/ipra_clausal_canonical.zip`, same snapshot.

[5] ProveIt contributors. *Reflexive base.* `docs/incoming/ipra_reflexive_base.zip`, same snapshot.

[6] ProveIt contributors. *Scalar iteration uniqueness for iPRA.* `docs/incoming/ipra_scalar_uniqueness.zip`, same snapshot.

[7] ProveIt contributors. `Lean/ConciseIPRA/NativeEval.lean`, `NativeZeroNormal.lean`, `NativeBiInterpretation.lean`, and `BiInterpretation.lean`, same project snapshot.

[8] ProveIt contributors. Five internal arithmetic and logical bridge fragments under `Research/certified_zero/reference/`, reproduced in the appendices with their license.

[9] ProveIt contributors. *Internal scalar predecessor bootstrap.* `Research/certified_zero/reference/scalar_predecessor.tex`, same snapshot; reproduced here.

[10] ProveIt contributors. *From literal-equation induction to full intuitionistic PRA.* `Research/certified_zero/reference/equality_reflection.tex`, same snapshot; reproduced here.

[11] ProveIt contributors. *Pairing, primitive recursion, and internal numerical composites.* `Research/certified_zero/reference/primitive_recursion.tex`, same snapshot; reproduced here.

Inherited bridge proofs

The following five source fragments are inherited reference material. Historical words such as “new” and “old” inside a fragment refer to that fragment’s original comparison, not to novelty claimed by this report. Its clause basis has both explicit Case and equation-progress choices; Section 4 explains their recovery in the selected five-pattern kernel. The entire dependency chain is included so the bi-interpretation claim does not rest on agreement over standard numerals.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$

the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* y B$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.

2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.

3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [9] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v+1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [10]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u,v,v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa,b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}(K(d(Sa,b))). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{40}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (25), which is equivalent to $d(Sa,b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (39) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, (a = b \vee (a = b \to \bot)) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [11]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \leq n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \leq i, \qquad R_n(i)^2 \leq n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \leq n < (q(n)+1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$
$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

F Complete raw root-free checker

The following listing is the actual standalone selected implementation, including parsing, validation, inference, and its public interface.

```
"""Raw clausal iPRA with computational equality.

Terms: 0, negative free levels, positive bound indices, [0,t] (S),
[n,a,h] (I). Formulas: [s,t] (equality), [B] (D).
Contexts contain (antecedent, one-bound-variable body) clauses.
Input: {'goal': a closed formula, 'proof': a finite proof tree}.
"""
import json
import sys


def fail():
    raise ValueError('invalid certificate')


def term(a, scope, depth=0):
    if type(a) is int:
        return -scope <= a <= depth
    return (type(a) is list and len(a) in (2, 3)
            and (len(a) == 3 or a[0] == 0)
            and all(term(t, scope, depth + (len(a) == 3 and i == 2))
                    for i, t in enumerate(a)))


def formula(a, scope, depth=0):
    return (type(a) is list
            and ((len(a) == 2 and all(term(t, scope, depth) for t in a))
                 or (len(a) == 1 and formula(a[0], scope, depth + 2))))


def opening(a, value, depth=0):
    if type(a) is int:
        return value if a == depth + 1 else a - (a > depth + 1)
    size = len(a)
    return [opening(t, value, depth + (2 if size == 1 else int(size == 3 and i == 2)))
            for i, t in enumerate(a)]


def derive(proof, context, scope):
    if type(proof) is not list or not proof or type(proof[0]) is not str:
        fail()
    tag = proof[0]
    if tag == 'u' and len(proof) == 4:
        index, witness, premise = proof[1:]
        if (type(index) is not int or not 0 <= index < len(context)
                or not term(witness, scope)):
            fail()
        antecedent, body = context[index]
        if derive(premise, context, scope) != antecedent:
            fail()
        return opening(body, witness)
    if tag == 'r' and len(proof) == 2:
        a = proof[1]
        if not term(a, scope):
            fail()
        return [a, a]
    if tag == 'v' and len(proof) == 3:
        s, t = derive(proof[1], context, scope)
        b, r = derive(proof[2], context, scope)
        c, a, h = b
        if c != 0:
            if len(c) != 2:
```

```
                fail()
            a, t = [c[1], a, h], opening(h, t)
        if a != s:
            fail()
        return [r, t]
    if tag in ('e', 'i') and len(proof) == 4:
        goal, data, branch = proof[1:]
        if not formula(goal, scope):
            fail()
        fresh = -scope - 1
        if tag == 'e':
            paired = derive(data, context, scope)
            if len(paired) != 1:
                fail()
            body = opening(paired[0], fresh, 1)
            clause = (opening(body, 0), body)
            target = goal
        else:
            if len(goal) != 1 or not term(data, scope):
                fail()
            body = opening(goal[0], data, 1)
            eta = fresh if len(goal[0]) == 2 and data == 0 else 0
            clause = ([0, 0], opening(body, eta))
            target = opening(body, [0, fresh])
        if derive(branch, context + [clause], scope + 1) != target:
            fail()
        return goal
    fail()


def verify(data):
    if type(data) is not dict or set(data) != {'goal', 'proof'}:
        fail()
    if not formula(data['goal'], 0):
        fail()
    if derive(data['proof'], [], 0) != data['goal']:
        fail()
    return True


if __name__ == '__main__':
    try:
        verify(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```

G Complete zero-normal companion checker

This separate implementation includes canonical syntax validation, normalizing opening, and the fixed-zero-base introduction. It uses the same external JSON wrapper.

```
"""Zero-normal clausal iPRA with uniform evaluation and zero-base introduction.

Terms: 0, negative free levels, positive bound indices, [0,t] (S),
[n,a,h] (I), n not literally 0. Formulas: [s,t] (equality), [B] (D).
Contexts contain (antecedent, one-bound-variable body) clauses.
Input: {'goal': a closed formula, 'proof': a finite proof tree}.
"""
import json
import sys


def fail():
    raise ValueError('invalid certificate')


def term(a, scope, depth=0):
    if type(a) is int:
        return -scope <= a <= depth
    return (type(a) is list and len(a) in (2, 3)
            and (len(a) == 2) == (a[0] == 0)
            and all(term(t, scope, depth + (len(a) == 3 and i == 2))
                    for i, t in enumerate(a)))


def formula(a, scope, depth=0):
    return (type(a) is list
            and ((len(a) == 2 and all(term(t, scope, depth) for t in a))
                 or (len(a) == 1 and formula(a[0], scope, depth + 2))))


def opening(a, value, depth=0):
    if type(a) is int:
        return value if a == depth + 1 else a - (a > depth + 1)
    size = len(a)
    a = [opening(t, value, depth + (2 if size == 1 else int(size == 3 and i == 2)))
         for i, t in enumerate(a)]
    return a[1] if size == 3 and a[0] == 0 else a


def derive(proof, context, scope):
    if type(proof) is not list or not proof or type(proof[0]) is not str:
        fail()
    tag = proof[0]
    if tag == 'u' and len(proof) == 4:
        index, witness, premise = proof[1:]
        if (type(index) is not int or not 0 <= index < len(context)
                or not term(witness, scope)):
            fail()
        antecedent, body = context[index]
        if derive(premise, context, scope) != antecedent:
            fail()
        return opening(body, witness)
    if tag == 'r' and len(proof) == 2:
        a = proof[1]
        if not term(a, scope):
            fail()
        return [a, a]
    if tag == 'v' and len(proof) == 3:
        s, t = derive(proof[1], context, scope)
        b, r = derive(proof[2], context, scope)
        c, a, h = b
        if len(c) != 2 or s != (a if c[1] == 0 else [c[1], a, h]):
```

```
            fail()
        return [r, opening(h, t)]
    if tag in ('e', 'i') and len(proof) == 4:
        goal, data, branch = proof[1:]
        if not formula(goal, scope):
            fail()
        fresh = -scope - 1
        if tag == 'e':
            paired = derive(data, context, scope)
            if len(paired) != 1:
                fail()
            body = opening(paired[0], fresh, 1)
            clause = (opening(body, 0), body)
            target = goal
        else:
            if len(goal) != 1 or not term(data, scope):
                fail()
            body = opening(goal[0], data, 1)
            base = opening(body, 0)
            clause = ([0, 0], opening(body, fresh) if base == [0, 0] else base)
            target = opening(body, [0, fresh])
        if derive(branch, context + [clause], scope + 1) != target:
            fail()
        return goal
    fail()


def verify(data):
    if type(data) is not dict or set(data) != {'goal', 'proof'}:
        fail()
    if not formula(data['goal'], 0):
        fail()
    if derive(data['proof'], [], 0) != data['goal']:
        fail()
    return True


if __name__ == '__main__':
    try:
        verify(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```